

Módulo 4. Empaquetado y serving con FastAPI y Docker

Curso MLOps/DataOps · ANBAN

José Picón

2026

1. Servir un modelo: qué significa y qué patrones existen

Hasta aquí tienes un modelo entrenado, versionado y registrado. Falta llevarlo a producción: que otra aplicación pueda hacerle preguntas y recibir respuestas. Eso se llama **serving**.

No todos los modelos se sirven igual. Hay cuatro patrones clásicos:

Patrón	Cuándo se usa	Ejemplo real
Batch	Predicciones masivas periódicas	Scoring nocturno de una cartera de clientes
Online	Respuesta en milisegundos por petición	Recomendador de productos en una web
Streaming	Datos llegando por una cola	Kafka + consumer que clasifica eventos en tiempo real
Edge	Sin red, en el propio dispositivo	Modelo de reconocimiento de voz en móvil

En este módulo vemos **online**, que es el más común y la base para muchos casos. Concretamente, vamos a servir el modelo `heart-failure-clf` del Lab 2 como una API REST.

2. Las librerías principales: FastAPI, Pydantic, Uvicorn

Estas tres librerías trabajan siempre juntas en Python moderno.

2.1 Qué es FastAPI

FastAPI es un framework para construir APIs REST en Python. Tres ventajas sobre alternativas más antiguas (Flask, Bottle):

1. **Rápido.** Se basa en Starlette, que usa `async/await`, y tiene rendimiento parecido a Node.js o Go.
2. **Validación automática.** Si declaras los tipos de los datos esperados, FastAPI los valida antes de que tu código los reciba.
3. **Documentación automática.** Genera una página interactiva en `/docs` sin que escribas nada extra.

2.2 Qué es Pydantic

Pydantic es la librería que FastAPI usa por debajo para validar datos. Defines una clase que describe el formato esperado del JSON de entrada y Pydantic:

- Convierte automáticamente tipos (string a int si tiene sentido).
- Rechaza datos que no encajan, devolviendo un error 422 al cliente.

Ejemplo de la API del Lab 3 (features clínicas):

```
from pydantic import BaseModel, Field

class Features(BaseModel):
    age: float = Field(ge=18, le=110)
    anaemia: int = Field(ge=0, le=1)
    ejection_fraction: int = Field(ge=5, le=90)
    serum_creatinine: float = Field(ge=0.1, le=20.0)
    sex: int = Field(ge=0, le=1)
    # ... y otras 6 features más
```

`Field(ge=18, le=110)` significa “greater or equal than 18, less or equal than 110”. Si un cliente manda `age=200`, Pydantic devuelve 422 y tu código nunca llega a ver ese dato malo.

2.3 Qué es Uvicorn

Uvicorn es el servidor que ejecuta tu aplicación FastAPI. Es a FastAPI lo que Gunicorn era a Flask: el motor que escucha peticiones HTTP y las pasa a tu código.

Para arrancar tu aplicación:

```
uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Lo que dice esto:

- `app.main:app` : importa el módulo `app/main.py` , dentro busca el objeto llamado `app` .
- `--host 0.0.0.0` : escucha en todas las interfaces de red.
- `--port 8000` : en el puerto 8000.

2.4 Instalación de las tres

En el lab del curso van instaladas dentro de la imagen Docker que construyes. Si quieres tenerlas en tu sistema:

```
pip install fastapi==0.115.4 "uvicorn[standard]==0.32.0" pydantic==2.9.2
```

El `[standard]` de uvicorn instala extras útiles (httptools, websockets, watchfiles para autoreload).

3. La otra parte: Docker, en serio

Hasta aquí Docker era una caja negra que arrancabas con `./setup.sh`. En este módulo vamos a construir **nuestra propia imagen**. Es el momento de entender el modelo.

3.1 Imagen vs contenedor

Las dos palabras se confunden todo el rato. La distinción es sencilla:

- **Imagen:** una plantilla congelada. Es como una receta + los ingredientes preparados. No se ejecuta.
- **Contenedor:** una imagen en ejecución. Es el plato cocinado a partir de la receta. Una imagen puede generar muchos contenedores.

3.2 El Dockerfile

Un **Dockerfile** es el fichero que describe cómo construir una imagen. Es una secuencia de instrucciones que parten de otra imagen (la “base”) y van añadiendo capas hasta obtener la imagen final.

Vamos a leer el Dockerfile del Lab 3 línea por línea:

```
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip wheel --no-cache-dir --wheel-dir /wheels -r requirements.txt
```

Etapas builder:

- `FROM python:3.11-slim AS builder` : parte de una imagen oficial de Python 3.11 en su versión “slim” (sin extras). El alias `builder` permite referirnos a esta etapa luego.
- `WORKDIR /build` : cambia a la carpeta `/build` dentro de la imagen.
- `COPY requirements.txt .` : copia el fichero `requirements.txt` del host al `/build/` de la imagen.
- `RUN pip wheel ...` : compila las dependencias a wheels (binarios precompilados) y los guarda en `/wheels`. Esto se hace en una etapa separada para no arrastrar a la imagen final las herramientas de compilación.

```

FROM python:3.11-slim AS runtime
WORKDIR /app

RUN apt-get update && apt-get install --no-install-recommends -y \
    curl \
    && rm -rf /var/lib/apt/lists/*

COPY --from=builder /wheels /wheels
COPY requirements.txt .
RUN pip install --no-cache-dir --no-index --find-links=/wheels -r
requirements.txt \
    && rm -rf /wheels

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    MLFLOW_TRACKING_URI=http://anban-mlflow:5000 \
    MODEL_URI=models:/heart-failure-clf/Staging

COPY app/ ./app/

RUN useradd -u 1000 -m svc && chown -R svc /app
USER svc

EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=3s --start-period=20s --retries=3 \

    CMD curl -fsS http://localhost:8000/health || exit 1

    CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--
workers", "2"]

```

Etapa runtime (la final):

- `FROM python:3.11-slim AS runtime` : parte otra vez de Python slim, pero en una imagen separada.
- `WORKDIR /app` : cambia a `/app`.
- `RUN apt-get ...` : instala `curl` (lo necesitamos para el healthcheck). El `rm -rf` borra los listados de paquetes para reducir el tamaño de la imagen.
- `COPY --from=builder /wheels /wheels` : trae los wheels precompilados desde la etapa builder.
- `RUN pip install ... --find-links=/wheels` : instala las dependencias desde los wheels locales (no descarga nada). Al final borra `/wheels` para ahorrar espacio.
- `ENV ...` : define variables de entorno por defecto. Pueden sobrescribirse al arrancar el contenedor con `-e`.
- `COPY app/ ./app/` : copia el código de la aplicación.
- `RUN useradd ... USER svc` : crea un usuario no privilegiado y se cambia a él. Por seguridad, los contenedores nunca deben correr como root.

- `EXPOSE 8000` : documenta que la aplicación escucha en 8000 (no abre el puerto realmente).
- `HEALTHCHECK ...` : define cómo Docker comprueba que la app está sana. Llama a `/health` cada 30 segundos.
- `CMD [...]` : el comando que se ejecuta cuando arranca el contenedor. Llama a `uvicorn` con la app.

3.3 Construir la imagen

Desde la carpeta `labs/lab3_serving/` :

```
docker build -t anban/income-api:lab3 .
```

Descomposición:

- `docker build` : instruye a Docker que construya una imagen.
- `-t anban/income-api:lab3` : le pone una etiqueta (nombre:versión).
- `.` : el contexto. Le dice a Docker que use el directorio actual, incluyendo el Dockerfile que hay aquí.

La primera vez tarda 1-3 minutos. Las siguientes, segundos, porque Docker cachea las capas.

3.4 Lanzar un contenedor

```
docker run -d \
  --name anban-income-api \
  --network docker_default \
  -p 8000:8000 \
  -e MLFLOW_TRACKING_URI=http://anban-mlflow:5000 \
  -e MLFLOW_S3_ENDPOINT_URL=http://anban-minio:9000 \
  -e AWS_ACCESS_KEY_ID=minio \
  -e AWS_SECRET_ACCESS_KEY=minio12345 \
  -e MODEL_URI=models:/heart-failure-clf/Staging \
  anban/income-api:lab3
```

Cada flag:

- `-d` : detached, en segundo plano.
- `--name anban-income-api` : nombre del contenedor, para referirnos a él luego (parar, ver logs, etc.).
- `--network docker_default` : lo metemos en la misma red que el resto del laboratorio. Así puede ver a `anban-mlflow` y `anban-minio` por su nombre.
- `-p 8000:8000` : publica el puerto. El primer 8000 es el del host, el segundo es el del contenedor.
- `-e VAR=value` : define una variable de entorno dentro del contenedor. Sobrescribe los defaults del Dockerfile.
- `anban/income-api:lab3` : la imagen que ejecutamos.

3.5 Comandos útiles para gestionar contenedores

Comando	Qué hace
<code>docker ps</code>	Lista los contenedores en ejecución.
<code>docker ps -a</code>	Lista todos los contenedores (incluso parados).
<code>docker logs anban-income-api</code>	Imprime los logs del contenedor.
<code>docker logs -f anban-income-api</code>	Igual pero “follow”, se queda esperando líneas nuevas.
<code>docker stop anban-income-api</code>	Para el contenedor.
<code>docker start anban-income-api</code>	Lo vuelve a arrancar.
<code>docker restart anban-income-api</code>	Stop + start.
<code>docker rm anban-income-api</code>	Borra el contenedor parado.
<code>docker rm -f anban-income-api</code>	Lo borra incluso si está corriendo.
<code>docker exec -it anban-income-api bash</code>	Entra al contenedor con una shell interactiva.
<code>docker images</code>	Lista las imágenes locales.
<code>docker rmi anban/income-api:lab3</code>	Borra una imagen.
<code>docker system prune -a</code>	Libera espacio borrando imágenes y contenedores no usados.

4. La aplicación: lectura del main.py

El fichero `labs/lab3_serving/app/main.py` define la API. Vamos a leerlo en orden de ejecución.

4.1 Imports y configuración

```
import os, time, uuid
from contextlib import asynccontextmanager
from typing import Any

import mlflow
import pandas as pd
from fastapi import FastAPI, HTTPException
from fastapi.responses import JSONResponse, PlainTextResponse
from pydantic import BaseModel, Field

MODEL_URI = os.environ.get("MODEL_URI", "models:/heart-failure-clf/Staging")

MLFLOW_URI = os.environ.get("MLFLOW_TRACKING_URI", "http://localhost:5000")
```

Lee dos variables de entorno. Si no están, usa defaults. La URI del modelo es lo que conecta esta API con el Model Registry del Lab 2.

4.2 Estado global y schema de entrada

```
state: dict[str, Any] = {
    "model": None,
    "run_id": None,
    "signature": None,
    "loaded_at": None,
    "predictions": 0,
    "errors": 0,
    "latency_sum_ms": 0.0,
}

class Features(BaseModel):
    age: float = Field(ge=18, le=110, description="Edad en años")
    anaemia: int = Field(ge=0, le=1, description="1 si hay anemia")
    creatinine_phosphokinase: int = Field(ge=1, le=12000,
        description="CPK en suero (U/L)")
    diabetes: int = Field(ge=0, le=1, description="1 si es diabético")
    ejection_fraction: int = Field(ge=5, le=90,
        description="Fracción de eyección ventricular (%)")
    high_blood_pressure: int = Field(ge=0, le=1)
    platelets: float = Field(ge=10000, le=1000000)
    serum_creatinine: float = Field(ge=0.1, le=20.0)
    serum_sodium: int = Field(ge=100, le=160)
    sex: int = Field(ge=0, le=1, description="0=mujer, 1=hombre")
    smoking: int = Field(ge=0, le=1)
```

- `state` es un diccionario donde guardamos el modelo cargado y contadores para métricas.
- `Features` es la clase Pydantic que describe el JSON de entrada. Cada campo lleva sus restricciones. FastAPI las usa para validar cada petición automáticamente.

4.3 Lifespan: cargar el modelo al arrancar

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    mlflow.set_tracking_uri(MLFLOW_URI)
    state["model"] = mlflow.pyfunc.load_model(MODEL_URI)
    md = state["model"].metadata
    state["run_id"] = md.run_id if md else None
    state["signature"] = md.signature if md else None
    state["loaded_at"] = time.time()
    yield
```

El **lifespan** es un hook de FastAPI que se ejecuta una vez al arrancar la app y otra al pararla. Aquí lo usamos para cargar el modelo una sola vez. Sin esto, lo cargarías en cada petición y la API sería 100 veces más lenta.

4.4 Endpoints

```
@app.get("/health")
def health():
    return {"status": "ok" if state["model"] is not None else "loading"}
```

`/health` lo usaría un load balancer para saber si esta réplica está sana. Si la app aún no ha cargado el modelo, devuelve “loading”.

```
@app.get("/version")
def version():
    return {
        "model_uri": MODEL_URI,
        "run_id": state["run_id"],
        "signature": str(state["signature"]),
    }
```

`/version` te dice qué modelo está sirviendo. Útil para auditoría rápida y para depurar despliegues.

```
@app.post("/predict", response_model=Prediction)
def predict(payload: Features) -> Prediction:
    rid = uuid.uuid4().hex[:12]
    t0 = time.perf_counter()
    X = _to_frame(payload)
    proba = float(state["model"].predict(X)[0])
    label = int(proba >= 0.5)
    dt = (time.perf_counter() - t0) * 1000
    return Prediction(label=label, proba=proba, ...)
```

`/predict` es el endpoint principal:

1. Pydantic ya validó el payload antes de llegar aquí. Si era inválido, FastAPI devolvió 422 sin tocar este código.
2. Genera un `request_id` único.
3. Convierte el payload a un DataFrame con el formato que espera el modelo (one-hot encoding).
4. Llama a `predict()` del modelo.
5. Aplica un umbral (0,5) para convertir probabilidad en etiqueta.
6. Devuelve la respuesta estructurada.

```
@app.get("/metrics", response_class=PlainTextResponse)
def metrics() -> str:
    return (
        f"# HELP api_predictions_total total predictions served\n"
        f"# TYPE api_predictions_total counter\n"
        f"api_predictions_total {state['predictions']}\n"
        ...
    )
```

`/metrics` devuelve métricas en formato Prometheus. Cualquier Prometheus puede hacer scraping de este endpoint y graficar las métricas en Grafana.

5. Antes de la práctica: requisitos

Para hacer este lab:

1. **Lab 2 completado:** tienes el modelo `heart-failure-clf` versión 1 en stage `Staging`.
2. **El laboratorio del curso arrancado.**
3. **Estar en `labs/lab3_serving/`.**

6. Ejercicio: empaquetar y servir el modelo

Objetivo del ejercicio: construir una imagen Docker con tu API, lanzarla, probar todos los endpoints con curl y comprobar que la validación Pydantic rechaza datos inválidos.

6.1 Paso 1 — Construir la imagen

```
docker build -t anban/income-api:lab3 .
```

Mientras compila, ves la lista de pasos del Dockerfile. La primera vez tarda 1-3 minutos. La imagen final pesa unos 1,2 GB (es grande porque incluye PyTorch y xgboost; en producción real usarías una imagen base más pequeña).

Verifica:

```
docker images anban/income-api
```

6.2 Paso 2 — Lanzar el contenedor

Antes de lanzar, asegúrate de que no haya un contenedor antiguo con el mismo nombre:

```
docker rm -f anban-income-api 2>/dev/null
```

Y lanzas:

```
docker run -d \
  --name anban-income-api \
  --network docker_default \
  -p 8000:8000 \
  -e MLFLOW_TRACKING_URI=http://anban-mlflow:5000 \
  -e MLFLOW_S3_ENDPOINT_URL=http://anban-minio:9000 \
  -e AWS_ACCESS_KEY_ID=minio \
  -e AWS_SECRET_ACCESS_KEY=minio12345 \
  -e MODEL_URI=models:/heart-failure-clf/Staging \
  anban/income-api:lab3
```

Espera unos 20-30 segundos a que arranque (tiene que cargar el modelo desde MinIO).

Verifica que está corriendo:

```
docker ps | grep anban-income-api
```

6.3 Paso 3 — Probar /health

```
curl -s http://localhost:8000/health
```

Salida esperada:

```
{"status": "ok", "uptime_s": 15}
```

Si todavía dice "loading", espera 10 segundos más. El modelo está descargándose desde MinIO.

6.4 Paso 4 — Probar /version

```
curl -s http://localhost:8000/version
```

Salida:

```
{
  "model_uri": "models:/heart-failure-clf/Staging",
  "run_id": "1944ae332b7249a9874dd98fd20c381a",
  "signature": "..."}

```

Anota el `run_id`: corresponde al run del Lab 2 que entrenó al ganador. Esa es la trazabilidad completa.

6.5 Paso 5 — Hacer una predicción real

```
curl -s -X POST http://localhost:8000/predict \
-H "content-type: application/json" \
-d '{
  "age": 75,
  "anaemia": 0,
  "creatinine_phosphokinase": 582,
  "diabetes": 0,
  "ejection_fraction": 20,
  "high_blood_pressure": 1,
  "platelets": 265000,
  "serum_creatinine": 1.9,
  "serum_sodium": 130,
  "sex": 1,
  "smoking": 0
}'
```

Estos son datos reales del primer paciente del dataset. Edad 75, fracción de eyección muy baja (20 %, lo normal es 55-70 %), creatinina alta (1,9 mg/dL, lo normal es < 1,3). Es un caso clínico de alto riesgo.

Respuesta:

```
{
  "label": 1,
  "proba": 1.0,
  "model_uri": "models:/heart-failure-clf/Staging",
  "request_id": "a3f8c2d4e9b0",
  "latency_ms": 4.21
}
```

`label=0` significa “no gana más de 50K”. `proba` es la probabilidad que el modelo asigna a la clase positiva. `latency_ms` mide cuánto tardó la inferencia.

6.6 Paso 6 — Probar la validación con un dato malo

```
curl -s -X POST http://localhost:8000/predict \
-H "content-type: application/json" \
-d '{
  "age": 200,
  "anaemia": 0,
  "creatinine_phosphokinase": 100,
  "diabetes": 0,
  "ejection_fraction": 35,
  "high_blood_pressure": 0,
  "platelets": 250000,
  "serum_creatinine": 1.0,
  "serum_sodium": 140,
  "sex": 1,
  "smoking": 0
}'
```

Pydantic debería rechazar el `age=200` con un 422:

```
{
  "detail": [
    {
      "type": "less_than_equal",
      "loc": ["body", "age"],
      "msg": "Input should be less than or equal to 110",
      "input": 200
    }
  ]
}
```

El modelo no se llega a ejecutar: la validación lo paró antes.

6.7 Paso 7 — Probar el endpoint de métricas

```
curl -s http://localhost:8000/metrics
```

Salida en formato Prometheus:

```
# HELP api_predictions_total total predictions served
# TYPE api_predictions_total counter
api_predictions_total 1
# HELP api_errors_total total errors
# TYPE api_errors_total counter
api_errors_total 0
# HELP api_latency_avg_ms average inference latency
# TYPE api_latency_avg_ms gauge
api_latency_avg_ms 4.21
```

6.8 Paso 8 — Probar la documentación automática

Abre <http://localhost:8000/docs> en el navegador. Verás la documentación Swagger generada automáticamente por FastAPI. Puedes hacer las pruebas anteriores desde el navegador con el botón “Try it out”.

6.9 Paso 9 — Ver los logs

```
docker logs anban-income-api
```

Verás líneas como:

```
INFO:      Started server process [7]
INFO:      Waiting for application startup.
loading model from models:/heart-failure-clf/Staging
model loaded · run_id=...
INFO:      Application startup complete.
INFO:      172.23.0.1:51920 - "POST /predict HTTP/1.1" 200 OK
INFO:      172.23.0.1:51924 - "POST /predict HTTP/1.1" 422 Unprocessable Entity
```

6.10 Paso 10 — Detener el contenedor

Cuando termines el lab:

```
docker stop anban-income-api
```

El contenedor se queda parado pero no se borra. Para volver a arrancarlo: `docker start anban-income-api`.

7. Conceptos de performance

Cuando despliegues una API real en producción, te van a hacer preguntas sobre rendimiento. Conviene tener el vocabulario claro.

Métrica	Qué significa
Latencia p50	La mitad de las peticiones tardan menos que esto.
Latencia p95	El 95 % tardan menos. Métrica habitual de SLA.
Latencia p99	El peor 1 % queda por debajo. Importante en escala.
Throughput (RPS)	Peticiones por segundo que aguanta sin perder calidad.
Cold start	Tiempo desde frío hasta la primera respuesta.

No te quedes con la media: una API con media de 100 ms y p99 de 5 s no es aceptable. Siempre mira la cola.

8. Checklist de cierre

- ¿Has construido la imagen `anban/income-api:lab3` ?
- ¿El contenedor está corriendo y responde a `/health` con `ok` ?
- ¿ `/version` devuelve el `run_id` correcto?
- ¿ `/predict` devuelve una respuesta estructurada con `label`, `proba`, `request_id` y `latency_ms`?
- ¿La validación Pydantic ha devuelto 422 con `age=200` ?
- ¿Has visto las métricas en formato Prometheus en `/metrics` ?

Si todo es sí, pasas al módulo 5 (CI/CD y drift).

9. Para profundizar

- **FastAPI documentation:** <https://fastapi.tiangolo.com>. La sección “Tutorial - User Guide” es excelente.
- **Pydantic v2:** <https://docs.pydantic.dev>. Para validaciones más complejas.
- **Docker docs:** <https://docs.docker.com>. El “Get Started” cubre todo lo que hemos visto.
- **BentoML:** <https://docs.bentoml.org>. Alternativa especializada en servir modelos ML, con batching dinámico y otras optimizaciones.